

Enterprise DevSecOps pip-audit Summary

Python Dependency Vulnerability Assessment and Remediation Report

Analyst	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Assessment Area	Software Composition Analysis - Python dependencies
Primary Tool	pip-audit
Evidence Sources	Local scan report, GitHub Actions job output, requirements.txt, JSON artifact

3	5	25	21
Top-level packages scanned	Affected packages	Raw findings	Unique advisories

This document summarizes dependency risk identified by pip-audit in the deliberately vulnerable Flask application used in the Enterprise DevSecOps Security Lab. The report explains scan scope, affected packages, evidence, business impact, recommended upgrades, CI/CD integration, and retest criteria.

Report Highlights	SCA scan scope, package-level findings, CI/CD evidence, remediation plan, secure requirements baseline, and retest criteria.
-------------------	--

Document Classification: Portfolio / Private Lab Evidence

1. Executive Summary

This pip-audit summary documents the Python dependency vulnerability assessment performed for the Enterprise DevSecOps Security Lab. The assessed Flask application intentionally included vulnerable dependency versions to demonstrate Software Composition Analysis (SCA), dependency risk triage, remediation planning, and CI/CD evidence generation.

The scan identified 25 raw vulnerability records across 5 affected packages. After de-duplication, the report contains 21 unique advisories. The most significant dependency risk is concentrated in the old requests dependency chain, which pulled vulnerable versions of urllib3 and idna. Additional findings were identified in gunicorn and Flask.

Assessment Objectives

- Identify known vulnerabilities in Python packages used by the demo application.
- Demonstrate dependency scanning locally and inside the CI/CD workflow.
- Translate pip-audit output into professional findings, risk themes, and remediation actions.
- Define a safe upgrade path and retest criteria for dependency hardening.
- Produce recruiter-friendly evidence showing SCA skill within a broader DevSecOps project.

Dependency Scope

Package	Pinned Version	Role in Application
Flask	2.3.3	Web application framework for the vulnerable demo app.
requests	2.19.1	HTTP client library intentionally pinned to an old version to create SCA findings.
gunicorn	21.2.0	Production-style WSGI server used by the hardened Docker image.

Command Evidence

```
pip-audit -r src/app/requirements.txt -f json -o security/reports/pip-audit-report.json || true
```

The || true control was used in the lab pipeline so that vulnerability evidence could be collected and uploaded as an artifact even when findings were intentionally present.

2. Scan Results Overview

Metric	Value	Interpretation
Top-level packages in requirements.txt	3	Flask, requests, and gunicorn were directly pinned.
Affected packages in resolved environment	5	Includes transitive dependencies pulled by top-level packages.
Raw vulnerability records	25	Total records reported by pip-audit JSON output.
Unique advisories	21	De-duplicated by package, advisory ID, and fix version set.
Highest remediation priority	High	Requests, urllib3, and gunicorn require priority upgrades.

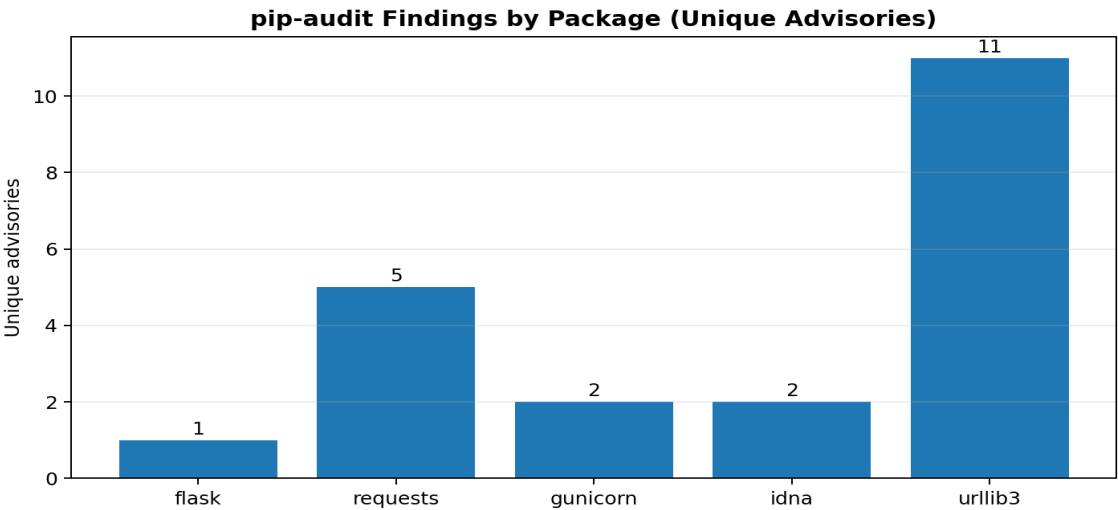


Figure 1: Unique pip-audit advisories grouped by affected package.

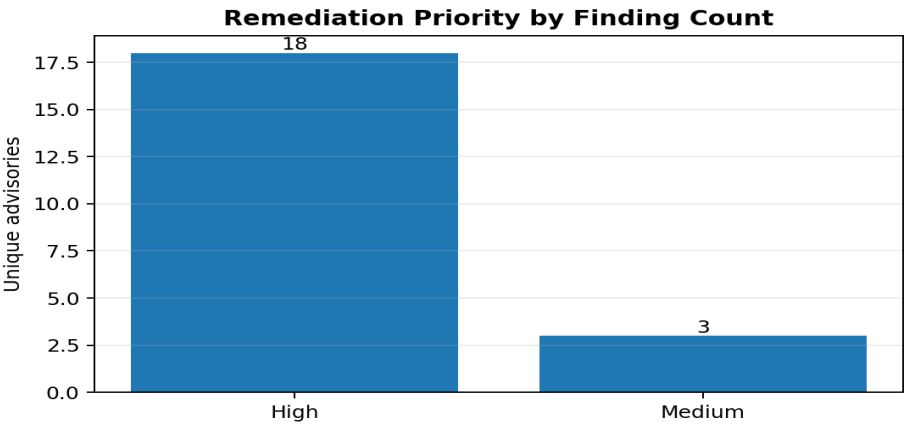


Figure 2: Remediation priority distribution derived from package exposure and advisory concentration.

3. Package-Level Finding Analysis

pip-audit does not only identify direct dependencies. It also resolves dependency relationships and reports vulnerabilities in transitive packages. In this project, the old `requests==2.19.1` pin caused an outdated dependency chain that included vulnerable `urllib3==1.23` and `idna==2.7`.

Package	Unique Advisories	Priority	Fix Version Guidance
requests	5	High	2.20.0, 2.31.0, 2.32.0, 2.32.4
urllib3	11	High	1.24.2, 1.24.3, 1.25.9, 1.26.17, 2.0.6
gunicorn	2	High	22.0.0
idna	2	Medium	3.15, 3.7
flask	1	Medium	3.1.3

Finding 1: requests 2.19.1 - outdated HTTP client dependency

The `requests` package was pinned to version 2.19.1, which is significantly outdated. pip-audit identified multiple advisories affecting redirect handling, credential leakage conditions, certificate verification behavior, and related HTTP client risk patterns. Since `requests` is a high-usage library, vulnerabilities in this dependency can affect outbound API calls, integrations, and service-to-service communication.

- Risk theme: sensitive header exposure, redirect handling issues, and insecure HTTP client behavior.
- Primary remediation: upgrade `requests` to at least the highest fixed version required by the report, then retest.
- Recommended target for this lab: `requests>=2.33.0,<3`.

Finding 2: urllib3 1.23 - vulnerable transitive dependency

The scan showed the largest advisory concentration in `urllib3`. In this lab, `urllib3` was not directly pinned in `requirements.txt`; it was introduced by the old `requests` dependency chain. This is important because it demonstrates why SCA must inspect both direct and transitive dependencies rather than only reading the top-level requirements file.

- Risk theme: HTTP request handling, redirect behavior, response parsing, and network-layer dependency risk.
- Primary remediation: upgrade `requests` so that a supported `urllib3` version is resolved; use constraints if needed.
- Recommended target for this lab: `urllib3>=2.7.0` where compatible, or use the secure version selected by updated `requests`.

Finding 3: gunicorn 21.2.0 - WSGI server advisory exposure

`gunicorn` is the runtime WSGI server used to run the Flask application in the hardened Docker image. pip-audit identified advisories fixed in version 22.0.0. Because `gunicorn` handles HTTP request processing at runtime, it should be treated as a production-facing dependency even in a lab container.

- Risk theme: web server request handling behavior and runtime exposure.
- Primary remediation: upgrade `gunicorn` to at least 22.0.0 and rerun application smoke tests.
- Recommended target for this lab: `gunicorn>=22.0.0,<23`.

4. Additional Dependency Findings and Impact

Finding 4: idna 2.7 - outdated transitive dependency

The idna package was resolved as version 2.7 and was flagged by pip-audit. This package is commonly used in URL and domain-name handling. Although it may not be directly imported by the application code, it is part of the HTTP client dependency chain and should be updated through the requests upgrade path.

- Risk theme: domain parsing and URL handling dependency risk.
- Primary remediation: allow updated requests dependencies to resolve a safe idna version.
- Recommended target for this lab: idna>=3.15 where compatible.

Finding 5: Flask 2.3.3 - framework advisory

Flask was flagged for an advisory with a fixed version indicated by the scan output. The risk is context-dependent because it relates to application behavior around sessions and cache behavior. Even when a specific vulnerable code path is not actively used in the lab, framework-level findings should be remediated to keep the baseline current.

- Risk theme: framework behavior and potential cache/session handling exposure.
- Primary remediation: upgrade Flask to the fixed version specified by pip-audit.
- Recommended target for this lab: Flask>=3.1.3,<4.

Business and Security Impact

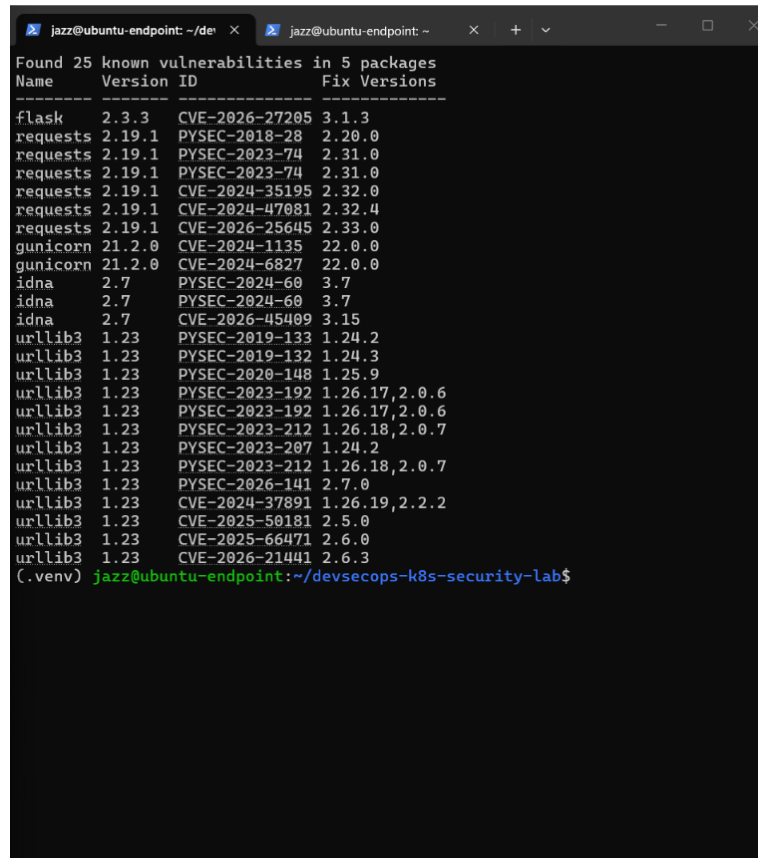
Impact Area	Explanation
Application Security	Known vulnerable packages increase the chance that a common library issue can be chained with application bugs such as command injection or SQL injection.
Container Security	Vulnerable Python dependencies are bundled into the Docker image, meaning the image inherits dependency risk until rebuilt with safe versions.
CI/CD Governance	Without SCA gates, vulnerable packages can continue moving through the delivery pipeline into deployment environments.
Compliance Readiness	Dependency scanning evidence supports secure SDLC, vulnerability management, and audit requirements.
Operational Risk	Outdated packages increase maintenance burden and can produce repeated security findings across environments.

Important Lab Context

The vulnerable dependency versions were intentionally included to prove that pip-audit can detect SCA issues and that the project can document remediation professionally. This report should be read as controlled lab evidence, not as evidence of a production system being exposed.

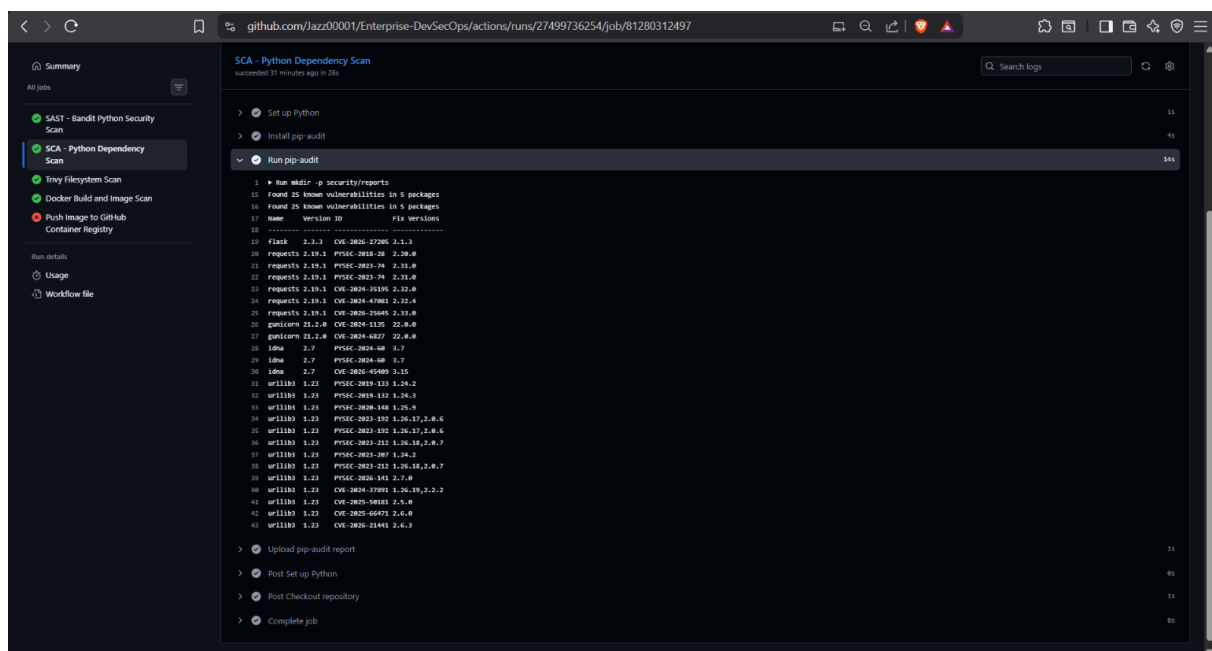
5. Evidence Review

This section records the two primary evidence sources used for this pip-audit summary: local terminal output and CI/CD job output. The local scan proves the tool was executed inside the private VM lab. The CI/CD screenshot proves dependency scanning was also integrated into the automated workflow.



```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ pip-audit
Found 25 known vulnerabilities in 5 packages
-----
Name      Version ID          Fix Versions
-----
flask      2.3.3   CVE-2026-27205  3.1.3
requests  2.19.1  PYSEC-2018-28   2.20.0
requests  2.19.1  PYSEC-2023-74   2.31.0
requests  2.19.1  PYSEC-2023-74   2.31.0
requests  2.19.1  CVE-2024-35195  2.32.0
requests  2.19.1  CVE-2024-47081  2.32.4
requests  2.19.1  CVE-2026-25645  2.33.0
gunicorn  21.2.0  CVE-2024-1135   22.0.0
gunicorn  21.2.0  CVE-2024-6827   22.0.0
idna       2.7     PYSEC-2024-60   3.7
idna       2.7     PYSEC-2024-60   3.7
idna       2.7     CVE-2026-45409  3.15
urllib3    1.23    PYSEC-2019-133  1.24.2
urllib3    1.23    PYSEC-2019-132  1.24.3
urllib3    1.23    PYSEC-2020-148  1.25.9
urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
urllib3    1.23    PYSEC-2023-207  1.24.2
urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
urllib3    1.23    PYSEC-2026-141  2.7.0
urllib3    1.23    CVE-2024-37891  1.26.19,2.2.2
urllib3    1.23    CVE-2025-50181  2.5.0
urllib3    1.23    CVE-2025-66471  2.6.0
urllib3    1.23    CVE-2026-21441  2.6.3
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 3: Local pip-audit dependency scan evidence from the private Ubuntu VM lab.



```
github.com/jazz0001/Enterprise-DevSecOps/actions/runs/27499736254/job/81280312497
SCA - Python Dependency Scan
succeeded 31 minutes ago in 26s

> Set up Python
> Install pip-audit
> Run pip-audit
1 Run mkdir -p security/reports
2 Found 25 known vulnerabilities in 5 packages
3 Found 25 known vulnerabilities in 5 packages
4 Name      Version ID          Fix Versions
5 -----
6 flask      2.3.3   CVE-2026-27205  3.1.3
7 requests  2.19.1  PYSEC-2018-28   2.20.0
8 requests  2.19.1  PYSEC-2023-74   2.31.0
9 requests  2.19.1  PYSEC-2023-74   2.31.0
10 requests  2.19.1  CVE-2024-35195  2.32.0
11 requests  2.19.1  CVE-2024-47081  2.32.4
12 requests  2.19.1  CVE-2026-25645  2.33.0
13 gunicorn  21.2.0  CVE-2024-1135   22.0.0
14 gunicorn  21.2.0  CVE-2024-6827   22.0.0
15 idna       2.7     PYSEC-2024-60   3.7
16 idna       2.7     PYSEC-2024-60   3.7
17 idna       2.7     CVE-2026-45409  3.15
18 urllib3    1.23    PYSEC-2019-133  1.24.2
19 urllib3    1.23    PYSEC-2019-132  1.24.3
20 urllib3    1.23    PYSEC-2020-148  1.25.9
21 urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
22 urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
23 urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
24 urllib3    1.23    PYSEC-2023-207  1.24.2
25 urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
26 urllib3    1.23    PYSEC-2026-141  2.7.0
27 urllib3    1.23    CVE-2024-37891  1.26.19,2.2.2
28 urllib3    1.23    CVE-2025-50181  2.5.0
29 urllib3    1.23    CVE-2025-66471  2.6.0
30 urllib3    1.23    CVE-2026-21441  2.6.3
31
32 > Upload pip-audit report
33
34 > Post Set up Python
35
36 > Post Checkout repository
37
38 > Complete job
```

Figure 4: GitHub Actions pip-audit job output evidence from the CI/CD workflow.

6. Remediation Plan

The remediation strategy is to upgrade vulnerable direct dependencies first, rebuild the environment, allow compatible transitive dependencies to update, then rerun pip-audit and the application test suite. Because the risk is concentrated in an old requests dependency chain, updating requests is the highest-value action.

Recommended Secure requirements.txt Baseline

```
Flask>=3.1.3,<4
requests>=2.33.0,<3
unicorn>=22.0.0,<23
```

Remediation Workplan

Phase	Action	Expected Outcome
1. Upgrade direct dependencies	Update Flask, requests, and gunicorn according to fixed versions from pip-audit.	Direct dependency risk reduced.
2. Rebuild environment	Recreate virtual environment and rebuild Docker image using the updated requirements file.	Container contains patched Python packages.
3. Retest application	Run health check, route smoke tests, and CI pipeline jobs.	Functional behavior remains stable after upgrades.
4. Rerun SCA	Run pip-audit locally and in GitHub Actions.	New report confirms reduced or zero known vulnerabilities.
5. Enforce pipeline gate	Set the dependency scan job to fail on high-risk vulnerabilities after the lab demonstration phase.	Future vulnerable dependency changes are blocked early.

CI/CD Control Recommendation

During initial evidence collection, the pipeline allowed findings to continue so reports could be uploaded. After remediation, the dependency scanning job should be changed from evidence mode to enforcement mode by removing the permissive `|| true` behavior for release branches.

7. Retest Criteria and Governance

Retest Procedure

- Update src/app/requirements.txt with the secure dependency baseline.
- Delete and recreate the Python virtual environment to avoid stale packages.
- Run pip install -r src/app/requirements.txt and confirm resolved package versions.
- Run pip-audit -r src/app/requirements.txt and export a new JSON report.
- Run docker build and Trivy image scan to validate the patched image.
- Execute Flask health-check and route smoke tests to confirm application stability.
- Push changes to GitHub and confirm the dependency scan job passes in the CI/CD workflow.

Validation Commands

```
python -m pip install --upgrade pip
pip install -r src/app/requirements.txt
pip-audit -r src/app/requirements.txt -f json -o security/reports/pip-audit-remediated-report.json
docker build -t devsecops-vuln-app:remediated .
trivy image devsecops-vuln-app:remediated
```

Governance Recommendations

Control	Recommendation
Dependency pinning	Use upper and lower version bounds to avoid both stale packages and uncontrolled major upgrades.
Automated scanning	Run pip-audit on every pull request and push to protected branches.
Release gate	Fail release builds when high-risk vulnerabilities have fixed versions available.
Artifact retention	Upload JSON and text reports as CI artifacts for auditability.
Exception handling	Document accepted risk with owner, expiration date, and compensating control.

Appendix A: Unique Advisory Inventory

The table below summarizes the de-duplicated pip-audit advisory inventory used in this report. Long advisory descriptions were intentionally omitted from the table to preserve readability; the complete JSON output remains available in `security/reports/pip-audit-report.json`.

Package	Installed	Advisory ID	Aliases	Fix Version(s)
flask	2.3.3	CVE-2026-27205	GHSA-68rp-wp8r-4726	3.1.3
unicorn	21.2.0	CVE-2024-1135	GHSA-w3h3-4rj7-4ph4	22.0.0
unicorn	21.2.0	CVE-2024-6827	GHSA-hc5x-x2vx-497g	22.0.0
idna	2.7	CVE-2026-45409	GHSA-65pc-fj4g-8rjx	3.15
idna	2.7	PYSEC-2024-60	GHSA-jjg7-2v4v-x38h, CVE-2024-3651	3.7
requests	2.19.1	CVE-2024-35195	GHSA-9wx4-h78v-vm56	2.32.0
requests	2.19.1	CVE-2024-47081	GHSA-9h9g-9r4m-mvj7	2.32.4
requests	2.19.1	CVE-2026-25645	GHSA-gc5v-m9x4-r6x2	2.33.0
requests	2.19.1	PYSEC-2018-28	CVE-2018-18074, GHSA-x84v-xcm2-53pg	2.20.0
requests	2.19.1	PYSEC-2023-74	CVE-2023-32681, GHSA-j8r2-6x86-q33q	2.31.0
urllib3	1.23	CVE-2024-37891	GHSA-34jh-p97f-mpxf	1.26.19, 2.2.2
urllib3	1.23	CVE-2025-50181	GHSA-pq67-6m6q-mj2v	2.5.0
urllib3	1.23	CVE-2025-66471	GHSA-2xpw-w6gg-jr37	2.6.0
urllib3	1.23	CVE-2026-21441	GHSA-38jv-5279-wg99	2.6.3
urllib3	1.23	PYSEC-2019-132	CVE-2019-11236, GHSA-r64q-w8jr-g9qp	1.24.3
urllib3	1.23	PYSEC-2019-133	GHSA-mh33-7rrq-662w, CVE-2019-11324	1.24.2
urllib3	1.23	PYSEC-2020-148	GHSA-wqvq-5m8c-6g24, CVE-2020-26137	1.25.9
urllib3	1.23	PYSEC-2023-192	GHSA-v845-jxx5-vc9f, CVE-2023-43804	1.26.17, 2.0.6
urllib3	1.23	PYSEC-2023-207	CVE-2018-25091, GHSA-gwvm-45gx-3cf8	1.24.2
urllib3	1.23	PYSEC-2023-212	GHSA-g4mx-q9vg-27p4, CVE-2023-45803	1.26.18, 2.0.7
urllib3	1.23	PYSEC-2026-141	CVE-2026-44431, GHSA-qccp-gfcp-xxvc	2.7.0

Appendix B: Report Conclusion

The pip-audit assessment successfully demonstrated Software Composition Analysis within the Enterprise DevSecOps Security Lab. The project shows how vulnerable dependencies can be identified, documented, prioritized, remediated, and validated through both local execution and CI/CD automation.

Key Takeaways

- A small requirements file can still produce many vulnerability findings because of transitive dependencies.
- Old HTTP client stacks such as requests 2.19.1 can bring vulnerable urllib3 and idna versions into the runtime environment.
- Dependency scanning should be treated as a pipeline control, not a one-time manual activity.
- Remediation should include functional testing, container rebuild, image scanning, and CI validation.
- Security reports should preserve both technical detail and clear remediation ownership.

Portfolio Value

This document supports the overall project by proving that dependency vulnerability management was performed professionally. It also demonstrates an ability to convert raw scanner output into a clear, recruiter-friendly report with evidence, impact analysis, remediation guidance, and validation criteria.

Final Status: Findings documented with remediation path and retest criteria defined.